

Getting Started with Math 4300

There are a few things that you need to do to prepare for Math 4300. Try to do all of these before the first day of class, but it's okay if you haven't done them by then. If you run into any technical issues, we can discuss them in class.

LeetCode

LeetCode (leetcode.com) is a website that provides lots of programming practice problems in a variety of programming languages, typically at a difficulty level that you might encounter in a job interview. (Which means that they're also a good difficulty level for a test problem in this course!) We'll use it for many of our in-class exercises. You should register for a LeetCode account.

You may want to attempt an easy LeetCode problem in a language that you already know (e.g., Python) to get used to how it works.

The Terminal

The terminal is a text-based interface to interact with the computer. On MacOS/Linux, this should be **Terminal**; on Windows, you should have **PowerShell**.

Software engineers need to know how to work with the terminal. To practice that, we'll set up a folder to store all your Math 4300 work. First, open the terminal. Then, type `cd` (which stands for "change directory") followed by the directory where you want to set up your Math 4300 folder. For example, I typed:

```
cd C:/Users/James/OneDrive/Desktop/Teaching_Materials
```

Next, create your Math 4300 directory by using `mkdir` ("make directory") followed by the name you wish to give it, e.g.,

```
mkdir mth4300
```

Use `cd` again to navigate into your new directory:

```
cd mth4300
```

(Note that this time, we're navigating into a directory that's contained in our current directory, so we don't need to write out the full path to get there. If you want to go in the other direction – e.g., if I wanted to go from `mth4300` back into `Teaching_Materials` – you can use `cd ..` to do that. The abbreviation `..` can also be used with other terminal command that require a path.)

Now that you're inside your Math 4300 directory, let's create a C++ program named `hello_world.cpp`. On MacOS/Linux, you can do this with the command

```
touch hello_world.cpp
```

On Windows, use

```
New-Item hello_world.cpp
```

The `ls` command lists all files and folders in your current directory. If you enter it now, you should see that your directory contains one file, named `hello_world.cpp`.

Of course, you probably want to organize your Math 4300 directory into subdirectories, not leave all your work for the entire semester in one giant folder. Create a new directory `hello` within your Math 4300 directory. (Do you remember how to do that in the terminal?) Then you can copy `hello_world.cpp` into the folder `hello` using the `cp` (“copy”) command:

```
cp hello_world.cpp hello/hello_world.cpp
```

If you use `ls` again, you should now see that your Math 4300 directory contains a directory `hello` and `hello_world.cpp`. But we don’t need two copies of `hello_world.cpp`, so you can use the `rm` command to remove the one in the current directory:

```
rm hello_world.cpp
```

One more useful terminal command: if you want to know what directory you’re currently in, you can use `pwd` (“print working directory”). There’s a lot more to know about the terminal, but this is enough of an introduction for now.

IDE

In order to write code effectively, you’ll need an IDE (Integrated Development Environment). We’ll be coding in C++ in this course, so be sure to choose one that’s appropriate for C++. I’ll be using Visual Studio Code (commonly referred to as VS Code), but you’re welcome to choose your own. Whatever you choose, install it on your computer now, and make sure it’s set up to work with C++. If you’re using VS Code, that means that you’ll want the “C/C++” extension by Microsoft. You can find instructions on installing it here: [C/C++ for Visual Studio Code](#).

Once you have your IDE working, you can use it to edit `hello_world.cpp`. Paste the following code into the file:

```
#include <iostream>
int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Due to technical limitations with PDFs, the indentation will probably be missing if you copy-paste this code. You can put it back in manually if you want the code to look nicer, but the program will run without it.

If your IDE is set up correctly, it will (probably) automatically color-code the text to make the C++ syntax easier to follow. Remember to save the file afterward!

C++

Unlike Python, C++ is a compiled language, which means that a *compiler* needs to convert the human-readable source code into a set of instructions that the computer can execute. You can compile C++ programs using the `g++` command in the terminal. To check whether it’s installed, run

```
g++ --version
```

in your terminal. If this works, it should give you information about the version of `g++` installed on your computer; if it doesn't work, then `g++` is not installed. (If you installed it recently, you may need to close and re-open the terminal.)

If you need to install it, then on MacOS/Linux you can use the command

```
xcode-select --install
```

On Windows, follow the instructions in the article [C/C++ for Visual Studio Code](#) from the previous section. Make sure you follow the instruction about adding the path to MinGW to your PATH variables.

You can run `g++ --version` again to make sure your installation was successful.

Run `hello_world.cpp`

Now we can compile and run our Hello World program. Open the terminal and navigate to the directory that contains the course code, then run the command:

```
g++ -o hello_world -std=c++20 hello_world.cpp
```

If everything is set up correctly, this should create a program named `hello_world`, which you can run with the following command:

```
./hello_world
```

When testing your code, you may find it more convenient to run the code without leaving your IDE. For example, in VS Code you should have at the top-right of your screen a “Run or debug” button (with an icon of a play symbol and a bug). You can open the dropdown menu and click “Run Code” there.

AI

Your IDE probably came with some sort of automatic AI assistance. You should turn that **off** for this course. Leaving aside issues of academic integrity, you should want it off for your own good – you won't have access to an AI during exams, so you should practice working without an AI now. If students get help from AI without trying to work through problems on their own first, they are almost never well-prepared for exams.

If you're using VS Code, press `Ctrl` + `,` (Windows/Linux) or `Cmd` + `,` (macOS) to open Settings. In the search bar, paste `@id:chat.disableAIFeatures` and check the box to disable AI features.